

Guía de uso del evaluador de expresiones

Luciano David Duarte

ALP 2025 - Licenciatura en Ciencias de la Computación

1 Objetivo de la guía

Esta guía explica cómo compilar, ejecutar y probar el evaluador de expresiones desarrollado para el trabajo final. El programa permite escribir expresiones matemáticas en una variable principal x , evaluarlas en un punto y calcular su derivada mediante números duales.

La entrada textual se parsea a un árbol de expresiones, ese árbol puede optimizarse, evaluarse, imprimirse y derivarse automáticamente. Por eso la guía no sólo muestra comandos, sino también qué espera el programa en cada modo de uso.

2 Requisitos y ubicación

Para usar el proyecto hace falta tener GHC y Cabal instalados. Los comandos están pensados para ejecutarse desde la raíz del repositorio, por ejemplo:

```
cd C:\Users\lucho\ALP-2025-LCC
```

3 Compilación

El primer paso recomendado es compilar todo el proyecto:

```
cabal build all
```

Este comando compila la biblioteca, el ejecutable y lo necesario para los tests. Si Cabal responde **Up to date**, significa que no detectó cambios pendientes de compilación. Si hubo modificaciones en algún módulo, Cabal recompila sólo lo necesario.

4 Modo interactivo

El modo interactivo está pensado para probar una expresión por vez. Se inicia con:

```
cabal run ALP2025-LCC
```

El programa primero pide una expresión. Si el parser la acepta, muestra la expresión normalizada por el pretty printer y, cuando corresponde, también su versión optimizada. Después pide el valor de x y calcula dos resultados: el valor de la función y la derivada en ese punto.

```
>>> sin(x) + x^2
>> Ingrese el valor de x: 1
f(1) = 1.8414709848078965
f'(1) = 2.5403023058681398
```

El valor ingresado para x puede ser un número o una expresión constante como $\pi/2$. Para salir del modo interactivo se puede escribir **salir**, **quit** o **exit**.

5 Procesamiento de archivos

El modo archivo permite ejecutar varios casos en una sola corrida. Cada línea útil tiene el formato:

```
expresion @ valor_de_x
```

La parte izquierda se interpreta como expresión del DSL. La parte derecha se usa como valor de x . Por ejemplo:

```
x^2 + 2*x + 1 @ 3
sin(x) @ pi/2
log(x) + pi @ e
```

Para procesar un archivo se usa:

```
cabal run ALP2025-LCC -- -f examples\basico.txt
```

También existe la opción larga:

```
cabal run ALP2025-LCC -- --file examples\basico.txt
```

En algunas instalaciones de Windows puede fallar la ejecución directa del binario generado por Cabal por políticas del sistema. En ese caso se puede correr el mismo flujo con `runghc`:

```
cabal exec -- runghc -isrc -iapp app\Main.hs -f examples\basico.txt
```

6 Comentarios en archivos

Los archivos aceptan comentarios de línea y comentarios de bloque. Esto permite armar ejemplos explicativos sin que esas aclaraciones formen parte de la expresión.

```
-- Esto es un comentario
x^2 @ 3 -- comentario al final de la linea

{- Comentario
  de varias lineas -}

(x + {- comentario interno -} 1) * 2 @ 3
```

Los comentarios se limpian antes de parsear cada línea. Si una línea queda vacía después de quitar comentarios y espacios, se ignora.

7 Sintaxis del lenguaje

El lenguaje soporta los operadores aritméticos $+$, $-$, $*$, $/$ y $^$. El signo menos puede usarse como resta binaria o como menos unario. La precedencia que usa el parser es:

```
potencia > menos unario > multiplicacion/division > suma/resta
```

Esto significa que $-x^2$ se interpreta como $-(x^2)$. Si se quiere elevar el negativo completo, hay que escribir $(-x)^2$.

Las funciones soportadas son:

`sin, cos, tan`
`sinh, cosh, tanh`
`arsinh, arcosh, artanh`
`sqrt, exp, log`

También se reconocen las constantes `pi` y `e`. Estas constantes pueden aparecer tanto en la expresión como en el valor ubicado a la derecha de `@`.

8 Dominios y errores

El evaluador trabaja sobre números reales. Por eso algunas funciones tienen restricciones de dominio. `log(x)` requiere argumentos positivos y `sqrt(x)` no acepta valores negativos. Para las funciones inversas hiperbólicas, `arcosh(x)` requiere $x \geq 1$ y `artanh(x)` requiere $|x| < 1$. La tangente se rechaza en puntos donde el coseno es cercano a cero.

Cuando ocurre un problema, el programa devuelve un error controlado. Por ejemplo:

```
sqrt(-1) @ 0
```

produce:

```
[X] Error al evaluar: Error de dominio: sqrt requiere argumento no negativo
```

También se detecta división por cero:

```
1/0 @ 0
```

y variables no definidas:

```
y + 2 @ 1
```

El modo de usuario está centrado en `x`. Si aparece otra variable sin valor asociado en el entorno, se informa como variable no definida.

9 Optimización

Antes de evaluar, el programa aplica optimizaciones simples sobre el AST. Por ejemplo, puede transformar `x + 0` en `x`, `x * 1` en `x`, o calcular `2 + 3` como `5`. Estas reglas hacen que algunas expresiones se muestren de forma más limpia y prueban que el árbol puede transformarse antes de interpretarse.

La optimización es conservadora. No se aplican reglas como $x/x = 1$, $x*0 = 0$ o $x^0 = 1$ en todos los casos, porque podrían ocultar errores. Por ejemplo, si $x = 0$, la expresión x/x debe seguir dando división por cero. Esta decisión es importante: el optimizador no debe cambiar una expresión que falla por otra que parece válida.

10 Archivos de ejemplo

La carpeta `examples/` contiene archivos preparados para probar partes distintas del lenguaje. `basico.txt` muestra expresiones iniciales, `compuestas.txt` combina funciones, `constantes.txt` usa `pi` y `e`, `optimizaciones.txt` muestra reglas conservadoras y `casos_error.txt` fuerza errores controlados.

Una corrida simple para revisar el modo archivo es:

```
cabal exec -- runghc -isrc -iapp app\Main.hs -f examples\basico.txt
```

Otros archivos de la carpeta permiten revisar precedencia, derivación automática, optimización, errores de dominio, constantes y comentarios dentro de expresiones.

11 Pruebas

La suite completa se corre con:

```
cabal test
```

Si la ejecución del binario de test queda bloqueada, se puede usar:

```
cabal exec -- runghc -isrc test\TestSuite.hs
```

La suite actual contiene 138 casos. Además de probar evaluación básica, derivadas, parser y errores, incluye una verificación llamada:

Verificacion de Casos de ejemplo

Esa prueba lee todos los archivos `.txt` de `examples/`. La intención es que los ejemplos distribuidos con el proyecto también estén cubiertos por la validación automática.

12 Documentación

La documentación principal está en LaTeX. Los archivos fuente son:

```
docs/informe-final.tex
```

```
docs/guia-uso.tex
```